

SEN381 MILESTONE 1

PROJECT ENGINEERING DOCUMENT

GROUP E

Table of Contents

Document Control & Project Governance	3
Section 1: CivicConnect Problem Analysis & Business Need	5
1.1 Context of Existing Informal & Fragmented Workflows	5
1.2 Root-Cause Decomposition of the 8 Operational Breakdown Modes	6
1.3 Engineering Root Cause Analysis (5-Whys)	7
1.4 CivicConnect Engineering Value Proposition & Business Capabilities.....	7
Section 2: Stakeholder Analysis & Conflict Resolution	8
2.1 Multi-Dimensional Stakeholder Profiles	8
2.2 RACI Matrix.....	9
2.3 Power-Interest Grid	9
2.4 Inherent Conflict Surfaces & Engineering Trade-Off Resolutions.....	10
Section 3: Scope Baseline & Boundary Control.....	11
3.1 Product Scope vs. Project Scope.....	11
3.2 Committed In-Scope Functional Capabilities (Baseline Milestones 1–4).....	11
3.3 Deliberately Deferred Scope & Technical Justifications	12
3.4 Explicitly Out-of-Scope Exclusions & Non-Deliverables	12
3.5 Scope Baseline Protection & Master Brief Appendix E Change Management Protocol	12
Section 4: Baselined Requirements Specification & Acceptance Criteria	13
4.1 Requirements Engineering Principles & Taxonomy (ISO/IEC/IEEE 29148)	13
4.2 Prioritised Functional Requirements (FR-001 through FR-014)	13
4.3 Service Request Lifecycle Finite State Machine (FSM & Guard Matrix).....	15
4.4 Measurable Non-Functional Requirements Baseline (ISO/IEC 25010 Quality Model).....	15
Section 5: Constraints & Systemic Trade-Off Analysis	15
5.1 Explicit Project Constraint Boundaries (The 6 PMBOK Pillars)	15
Section 6: Initial Requirements Traceability Matrix (RTM v1.0) (Output 6)	16
6.1 Traceability Methodology & The Lifecycle Chain (Intent -> Evidence)	16
6.2 RTM Structural Schema (Stakeholder -> Req -> Design -> PR -> Test -> Evidence)	16
6.3 Live Milestone 1 Requirements Traceability Matrix Table (FR-001–014 & NFR-001–010)....	16
6.4 End-to-End Traced Demonstration Walkthrough (Deep Trace of FR-010)	18
Section 7: Initial Project Risk Register & Uncertainty Management (Output 7)	18
7.1 Risk Assessment Methodology & Quantitative Exposure Formula ($RE = P \times I$).....	18
7.2 Live Project Risk Register Table (RSK-001 through RSK-010)	18

Section 8: Forward Engineering Considerations Register	18
8.1 The Strategic Engineering Horizon: Thinking Ahead vs. Premature Coding.....	18
8.2 Register of 6 Strategic Project-Specific Lifecycle Concerns	19
8.3 Justified Decision Deferment Defence.....	22
Section 9: Engineering Decision Log & Architecture Decision Records (ADRs)	22
9.1 Master Engineering Decision Table (Context, Trade-offs & Consequences).....	22
9.2 Formal Milestone 1 ADRs:.....	23
Section 10: Deployment, Environments & Operational Readiness Concept	25
10.1 The Four-Tier Environment Separation Model.....	25
10.2 Environment Parity & Local Containerized Replication	26
10.3 Deployment vs Release Distinction & Rollback Strategy	26
10.4 Operational Health Probes & Structured JSON Telemetry	27
Section 11: Milestone 1 Baseline Sign-Off Gate & Governance Audit	28
11.1 Master Project Brief Appendix D Verification Table	28
11.2 Gate Review Summary & Readiness Declaration.....	29
11.3 Formal Sign-Off Record & Milestone Gate Outcome (ACCEPTED)	29
Section 12: Academic Reference List.....	30
12.1 International Software Engineering Standards (IEEE, ISO/IEC)	30
12.2 Architectural & Methodological Literature (Bass et al., Boehm, Fowler).....	30
12.3 Statutory Legislation & Security Frameworks (POPIA, NIST, OWASP).....	30
Section 13: Appendices.....	31
Appendix A: Master Project Brief Appendix D Baseline Sign-Off Table.....	31
Appendix B: Requirements Traceability Matrix (RTM v1.0)	31
Appendix C: Project Risk Register (RSK-001 to RSK-010)	31
Appendix D: Team Working Agreement & Collaboration SLA	31
Appendix E: Responsible AI Usage Register v1.0 & Case Critique	32

Document Control & Project Governance

0.1 Master Baseline Information

- **Project Name:** CivicConnect (Community Service Request Management Platform)
- **Document Identifier:** SEN381_Milestone1_GroupE
- **Baseline Type:** Milestone 1 – Engineering Foundation & Requirements Baseline
- **Current Baseline State:** v1.0 (Controlled Product State)

- **Configuration Repository:** <https://github.com/SEN381-CivicConnect/CivicConnect.git>

0.2 Authorship, Review & Approval Record

Version	Date	Primary Author(s)	Peer Reviewer 1 (Sign-off)	Peer Reviewer 2 (Sign-off)	Baseline Status	Summary of Milestone 1 Changes
0.1	2026-09-01	Pandora Greyling	Lisa Verson	Chris Fourie	Initial Working Draft	Initial project structure, problem context framing, and core sections.
0.5	2026-09-05	Lisa Verson & Chris Fourie	Pandora Greyling	Lisa Verson (Option 3 sections) / Chris Fourie (Option 1 sections)	Internal Review Baseline	Integrated requirements (FR/NFR), forward engineering considerations, and ADR drafts.
1.0	2026-09-08	Chris Fourie, Pandora Greyling & Lisa Verson	Pandora Greyling (Quality Lead)	Lisa Verson (Requirements Lead)	Controlled Baseline (APPROVED)	Formal M1 Engineering Baseline: Sections 1–12 complete, RTM, Risk Register, Decision Log, 4-Tier Deployment Concept, and Gate Sign-off. <i>(Note: Chris Fourie approved Option 1 & 2 sections; Pandora & Lisa approved Option 3 sections).</i>

0.3 Registered Project Team & Governance Specialization

Student ID	Full Name	Designated Role	Primary Milestone Ownership
602006	Lisa Verson	Lead Requirements Analyst	Problem analysis, stakeholder conflict resolution, Functional Requirements (FR-001 to FR-014), Gherkin acceptance criteria, and Scope Baseline statement.
602369	Pandora Greyling	Quality & Risk Manager	Non-Functional Requirements (NFR-001 to NFR-010), dynamic constraints & trade-off analysis, Project Risk Register (RSK-001 to RSK-010), and testability foundation.
602826	Chris Fourie	Systems Architect & Governance Lead	Document Control & Governance (Document Header, Tables 0.1–0.3). Section 8: Forward Engineering Considerations Register (FEC-001 to FEC-006). Section 9: Engineering Decision Log & Formal Architecture Decision Records (ADR-001, ADR-002, ADR-003). Section 10: 4-Tier Deployment, Local Parity & Operational Readiness Concept. Section 11: Formal Baseline Sign-Off Gate (Master Project Brief Appendix D). Section 12: Academic Reference List (Harvard Style). Standalone Governance Artefacts: GitHub Repository Governance & PR Templates, AI Usage Register v1.0, and Team Working Agreement. Presentation & Defence Lead

Section 1: CivicConnect Problem Analysis & Business Need

1.1 Context of Existing Informal & Fragmented Workflows

The organisation currently manages community service requests (facility faults, damaged equipment, security concerns, IT support, maintenance issues, lost property, and other operational requests) through an uncoordinated mix of WhatsApp messages, spreadsheets, email, telephone calls and paper records. This informal, multi-channel approach was adequate when request volumes were low, but it has become a structural

liability: no channel is authoritative, no record persists reliably, and no single party owns the lifecycle of a request from submission to resolution.

CivicConnect exists to replace this fragmented process with a single controlled digital platform that makes the full lifecycle of a service request submission, categorisation, assignment, action, resolution and closure visible, traceable and auditable.

1.2 Root-Cause Decomposition of the 8 Operational Breakdown Modes

Failure Mode	Root Cause	Consequence
Requests duplicated, overlooked, misassigned or lost	No single record; each channel (WhatsApp/email/paper) is independent	Wasted staff effort, requester frustration, unresolved issues, repeated work on duplicate issues
Requesters have limited visibility of status	Lack of a collective database of requests and ability to correspond about their status.	Repeated follow-up calls/messages; erodes trust in the organisation
Staff struggle to prioritising requests, their ownership, and coordinating work.	No shared queue, no ownership field, no priority mechanism	Urgent requests delayed behind trivial ones, duplicated effort between staff
Weak accountability for status changes	No audit log of who changed what, when	Lack of clarity results in no basis for performance management and personal responsibility
Management lacks critical operational information (outstanding, overdue, and resolved work)	Data is scattered across spreadsheets kept by different individuals	Important decisions are made without crucial information, leaving these decisions prone to aimlessness
Reporting is laborious - manual, inconsistent, difficult to audit	Reports are hand-compiled from multiple disconnected sources	Inefficient and unnecessarily error-prone
Sensitive request information is handled inconsistently	WhatsApp/paper channels have no access control or retention policy	Exposure of privacy, POPIA non-compliance risk

No single controlled lifecycle record of service requests	Information is channel-bound, not request-bound	No traceability; impossible to audit or improve the process
---	---	---

1.3 Engineering Root Cause Analysis (5-Whys)

Why	Question	Answer
1	Why do requesters/management have no reliable visibility of request status?	There is no shared system tracking a request's current state.
2	Why is there no shared system tracking state?	Requests are logged ad hoc across disconnected channels (WhatsApp, email, paper, spreadsheets).
3	Why are these channels disconnected rather than unified?	No controlled platform was ever built to unify submission, assignment and lifecycle tracking.
4	Why was no such platform built previously?	Request volumes were historically low, so informal handling was seen as adequate and no tooling investment was made.
5	Why was this not revisited as volume and complexity grew?	There was no formal policy or trigger for evaluating operational tooling needs until the problems (duplication, lost accountability, poor reporting) had already accumulated.

Root Cause: the absence of a controlled, unified digital platform and of any governance trigger to invest in one rather than any single failure mode in isolation.

1.4 CivicConnect Engineering Value Proposition & Business Capabilities

CivicConnect's engineering value is not simply digitising a form. Its core value lies in establishing a single, controlled, traceable record for every service request, so that:

- Requesters gain visibility (failure modes 1–2) through a persistent, checkable status.
- Staff gain a coordinated, prioritizable queue with clear ownership (failure mode 3).

- Every status transition is attributable and time-stamped, restoring accountability (failure mode 4).
- Management gains a single source of truth for reliable, auditable reporting (failure modes 5–6).
- Access to sensitive request data is controlled and consistent, addressing privacy risk (failure mode 7).
- The full request lifecycle exists as one traceable record rather than fragments across channels (failure mode 8).

Section 2: Stakeholder Analysis & Conflict Resolution

2.1 Multi-Dimensional Stakeholder Profiles

Stakeholder Group	Description	Primary Concerns
Requesters	Community members that submit and track service requests	Ease of submission and visibility of status
Supervisors or Team Leaders	Assign, prioritize and coordinate the requests across staff	Coordination and workload balance
System Administrators	Admins in charge of configuring roles, access control and platform security	Secure configuration, system uptime and role-based access control integrity
External Stakeholders	Auditors, POPIA Compliance	Legal processing of personal data
Response/Operational Staff	Handles the submitted requests	Clear queue and enough detail provided to act accordingly
Management	Organisational leadership which overlooks performance and accountability	Performance of service, accountability and reliable data.

2.2 RACI Matrix

Activity	Requester	Staff	Supervisor	Management	Admins
Submit Request	R/A	I	-	-	-
Categorize Request	C	R/A	I	-	-
Assign/Accept Request	I	R	A	I	-
Update Request Status	I	R/A	C	I	-
View Reports or Dashboard	-	C	C	R/A	I
Configure roles and access control	-		C	A	R

2.3 Power-Interest Grid

KEEP SATISFIED (HIGH POWER, LOW INTEREST) System Administrators External Stakeholders	MANAGE CLOSELY (HIGH POWER, HIGH INTEREST) Management Supervisors/ Team leaders
MONITOR WITH MINIMUM EFFORT (LOW POWER, LOW INTEREST) Requestors	KEEP INFORMED (LOW POWER, HIGH INTEREST) Operational/Response Staff

2.4 Inherent Conflict Surfaces & Engineering Trade-Off Resolutions

2.4.1 Conflict Surface A: Anonymity vs POPIA Accountability

Tension: Requesters may want to report sensitive matters (e.g. security concerns) without full identification, while accountability and evidence trails (failure mode 4) require attributable data.

Resolution approach: Collect the minimum personal data necessary to action a request (data minimisation), restrict full identity visibility to authorised staff roles only, and log access to personal data separately from the request content itself.

Residual risk: A fully anonymous option cannot support follow-up or accountability; this trade-off must be documented and accepted by management as a deliberate scope decision, not silently defaulted.

2.4.2 Conflict Surface B: Staff Visibility vs Requester Privacy

Tension: Staff need enough request detail to act effectively, but broad staff visibility of all request content increases privacy exposure for requesters, especially for sensitive categories (e.g. harassment, security).

Resolution approach: Apply category-based and role-based access control (RBAC) so staff only see requests relevant to their category/team; sensitive categories may be restricted to a smaller authorised group.

Residual risk: Over-restriction could delay response to genuinely urgent sensitive requests; category/role boundaries must be reviewed periodically.

2.4.3 Conflict Surface C: Management Oversight vs Staff Autonomy

Tension: Management requires performance visibility (overdue/open/resolved metrics) to support accountability and reporting, but excessive granular monitoring of individual staff activity risks feeling like surveillance and can reduce trust and autonomy.

Resolution approach: Report at the level of request/category/team performance rather than individual real-time activity tracking; provide staff with visibility into their own metrics.

Residual risk: Aggregate reporting may obscure individual performance issues that management legitimately needs to identify; this is accepted as a deliberate scope boundary for the initial baseline.

Section 3: Scope Baseline & Boundary Control

3.1 Product Scope vs. Project Scope

Product Scope: the features and functions CivicConnect itself must deliver (the 14 in-scope features, FR-001–FR-014 and the NFR baseline).

Project Scope: the full body of controlled engineering work required to produce, evidence and defend that product: the PED, RTM, Risk Register, Decision Log, GitHub governance, AI Usage Register, stakeholder analysis and the milestone presentation/defence itself.

3.2 Committed In-Scope Functional Capabilities (Baseline Milestones 1–4)

3.2.1 Requester Capability Pillar (FR-001 to FR-005)

- Submit a new request with category with category, description and optional attachment
- Controlled category options for requests
- Requesters view current state of their requests
- Requesters view history of previously submitted request
- Requesters receive meaningful feedback or message when the status of their request is changed

3.2.2 Operational Staff Capability Pillar (FR-006 to FR-011)

- Staff view requests relevant to their role
- Staff can search, filter and sort requests by category, status or priority
- Authorized staff can view the full details of a request
- The ownership of a request can be accepted or assigned
- Request status will be updated through controlled transition (new – assigned – in progress- completed- closed)
- Comments, actions and resolution information of completed requests are logged

3.2.3 Management Oversight Capability Pillar (FR-012 to FR-014)

- Authorized roles can close or resolve requested that are completed
- Management can view useful service activity information and identify open, overdue, resolved and closed requests
- Role based access control is implemented for all above features.

3.3 Deliberately Deferred Scope & Technical Justifications

3.3.1 Multi-Channel Ingestion (SMS / WhatsApp Webhooks)

A WhatsApp bot would provide more convenience to requesters but would require additional security and external integration which cannot be justified within the required timeline.

3.3.2 Automated Escalation Engine

Changing the priority of a request automatically would provide a lot of value for accountability, but requires configurable business rules and additional testing so a manual alternative is acceptable for baseline 1

3.4 Explicitly Out-of-Scope Exclusions & Non-Deliverables

3.4.1 Real-Time messaging between requester and assigned staff

The core need for the requester is status visibility, adding live conversation may also impact the resolution speed of the request if the staff are flooded with messages.

3.4.2 Multi-Tenant Enterprise SaaS Partitioning

The brief scopes a single organisations service requests. Creating data isolation for multiple organisations that are going to use the app is not currently justifiable with no other organisations in consideration.

3.5 Scope Baseline Protection & Master Brief Appendix E Change Management Protocol

Once all sections are accepted at Milestone 1 baseline sign-off, this scope becomes protected: it may not be silently edited. Any proposed addition, removal or material change to scope after sign-off must be routed through the brief's Change Management Standard using the Appendix E Change Management Protocol

- No requirement is added, removed or reworded after baseline sign-off without an approved Change Request. this includes changes proposed informally in standups or chat.
- Every approved change updates the RTM, and where relevant the Risk Register and Decision Log, in the same commit/PR cycle as the change itself.
- Moving an item from Deferred or Out-of-Scope into In-Scope requires the same Impact analysis as adding a brand-new feature. Deferral/exclusion is not a lower bar for re-entry.
- If schedule pressure tempts the team to silently drop a Must-priority FR, this must instead be raised as a Change Request, so the trade-off is authorised and visible, not hidden
- The PED's version history preserves every prior baseline; approved changes are appended, never overwritten

Section 4: Baselined Requirements Specification & Acceptance Criteria

4.1 Requirements Engineering Principles & Taxonomy (ISO/IEC/IEEE 29148)

Requirement Level	Definition	CivicConnect Example
Business Requirement	High-level organisational need driving the project	"Reduce lost/duplicated service requests and restore accountability"
Stakeholder Requirement	What a specific stakeholder group needs the system to do for them	"Requesters need to see the status of a request they submitted"
System/Functional Requirement	A specific, verifiable capability the system shall provide	FR-003: "The system shall allow a Requester to view the current status of any request they submitted."
Non-Functional/Quality Requirement	A measurable constraint on how well a function performs	NFR-004: page load time for the requester status view
Design Constraint	A limitation imposed on the solution rather than an elicited need	RBAC must be enforced at the data layer, not only the UI layer

4.2 Prioritised Functional Requirements (FR-001 through FR-014)

ID	REQUIREMENT DESCRIPTION	SOURCE	MoSCoW
FR-001	System will allow a requester to submit a new request with a description and optional attachment	Requester	Must
FR-002	The system will allow the Requester to select a category from a controlled list	Requester	Must
FR-003	The system shall allow a Requester to view the current state of their requests	Requester	Must
FR-004	The system will notify a requester when their request status has	Requester	Should

	been updated (accepted, rejected, updated or completed)		
FR-005	The system will display a requestor's history of requests	Requester	Must
FR-006	The system display service requests to relevant staff	Response Staff/System Admin	Must
FR-007	The system will allow Staff to search filter or sort requests by category, status or priority	Response staff/ Supervisor	Must
FR-008	The system will allow authorized staff to view the full details of a request	Response Staff	Must
FR-009	The system will allow staff to accept or assign responsibility for a request	Response Staff, Supervisors	Must
FR-010	The system will update request status through controlled transitions (New to Assigned to In Progress etc.)	Response Staff, Supervisors	Must
FR-011	The system will allow staff to record relevant actions, comments or resolution information for a request.	Response Staff	Must
FR-012	The system will allow authorized roles to resolve or close a request	Response Staff, Supervisors	Must
FR-013	The system will provide management with a dashboard that summarizes requests by status (open, overdue, resolved or closed) and date	Management	Should
FR-014	The system will enforce role-based access control to ensure each user can	System Administrators	Must

	only view data and perform actions authorized for their role.		
--	---	--	--

4.3 Service Request Lifecycle Finite State Machine (FSM & Guard Matrix)

4.4 Measurable Non-Functional Requirements Baseline (ISO/IEC 25010 Quality Model)

4.4.1 Performance Efficiency (NFR-001 & NFR-007)

System should have minimum $\leq 500\text{ms}$ of latency for it to be considered performant

4.4.2 Reliability & Availability (NFR-002 & NFR-009)

System should have a $\geq 99\%$ uptime every year to be considered reliable.

System should be available within the required area

4.4.3 Usability & Accessibility (NFR-003: WCAG 2.1 Level AA)

System should be able to do any common action within 3-4 clicks.

System should comply with WCAG 2.1 level AA accessibility guidelines.

4.4.4 Security, Confidentiality & Privacy (NFR-004: RBAC & NFR-005: POPIA)

System should be protected from SQL attacks and implement two key authentication

System should handle all personal details within the POPIA act

4.4.5 Maintainability, Modularity & Testability (NFR-008: $\geq 80\%$ Coverage)

The system should have a $\geq 80\%$ test coverage

4.4.6 Auditability & Cost Sustainability (NFR-006 & NFR-010: \$0.00/Month)

System should use free tools that will not incur costs during daily use

Section 5: Constraints & Systemic Trade-Off Analysis

5.1 Explicit Project Constraint Boundaries (The 6 PMBOK Pillars)

5.1.1 Team Size Constraint (Fixed Capacity: Exactly 3 Students)

The team will have to work within the constraint of exactly 3 students not allowing any loss or gain of any team member.

5.1.2 Schedule Constraint (Fixed Academic Calendar across 4 Milestones)

The schedule is immutable and cannot be changed with different system requirements

5.1.3 Cost & Budget Constraint (Zero Budget: \$0.00 Free-Tier Cloud Hosting)

System should use free tier tools throughout the project these could create system lock in causing difficulty changing with increased budget

Section 6: Initial Requirements Traceability Matrix (RTM v1.0) (Output 6)

6.1 Traceability Methodology & The Lifecycle Chain (Intent -> Evidence)

1. First list which stakeholder owns this requirement
2. Then research and decide if the requirement is feasible
3. Make sure that the research is tested and reviewed
4. Accept or deny the new requirement

6.2 RTM Structural Schema (Stakeholder -> Req -> Design -> PR -> Test -> Evidence)

ID: creates a unique id that can be listed instead of the full requirement

Stakeholder: Lists the specific stakeholder that requires this item

Requirement: A description of the requirement

Design: Shows how this requirement is designed

Issue: The exact PR that is associated with this requirement

Test: The testing document that will be used to test the feature

Acceptance evidence: the document

6.3 Live Milestone 1 Requirements Traceability Matrix Table (FR-001-014 & NFR-001-010)

ID	Stakeholder	Requirement	Design	Issue	Test	Acceptance Evidence
FR-001	Requester	Submit service request	Deferred to milestone 2	Deferred to milestone 2		
FR-002	Requester	Categorize Requests	Deferred to milestone 2	Deferred to milestone 2		
FR-003	Requester	View Status of request	Deferred to milestone 2	Deferred to milestone 2		
FR-004	Requester	View history of requests	Deferred to milestone 2	Deferred to milestone 2		
FR-005	Requester	Receive	Deferred to milestone 2	Deferred to milestone 2		

		feedback to submitted requests				
FR-006	Staff	View Service request relevant to user	Deferred to milestone 2	Deferred to milestone 2		
FR-007	Staff/ Management	Search, Sort and filter requests	Deferred to milestone 2	Deferred to milestone 2		
FR-008	Staff	View request details	Deferred to milestone 2	Deferred to milestone 2		
FR-009	Staff	Assign requests to users	Deferred to milestone 2	Deferred to milestone 2		
FR-010	Staff	Update request status	Deferred to milestone 2	Deferred to milestone 2		
FR-011	Staff	Record details to in progress requests	Deferred to milestone 2	Deferred to milestone 2		
FR-012	Management	View service activity information	Deferred to milestone 2	Deferred to milestone 2		
FR-013	Management	Identify open, closed, resolved and overdue requests	Deferred to milestone 2	Deferred to milestone 2		
FR-014	Management	Service - Performance metrics accessible	Deferred to milestone 2	Deferred to milestone 2		
NFR-001	Standards	Performance	Deferred to milestone 2	Deferred to milestone 2		
NFR-002	Standards	Reliability	Deferred to milestone 2	Deferred to milestone 2		
NFR-003	Standards	Usability and accessibility	Deferred to milestone 2	Deferred to milestone 2		
NFR-004	Standards	security	Deferred to milestone 2	Deferred to milestone 2		
NFR-005	Standards	Confidentiality and Privacy	Deferred to milestone 2	Deferred to milestone 2		
NFR-006	Standards	Auditability	Deferred to milestone 2	Deferred to milestone 2		
NFR-007	Standards	Efficiency	Deferred to milestone 2	Deferred to milestone 2		
NFR-008	Standards	Maintainability	Deferred to milestone 2	Deferred to milestone 2		
NFR-009	Standards	Availability	Deferred to milestone 2	Deferred to milestone 2		

NFR-010	Standards	Cost sustainability	Deferred to milestone 2	Deferred to milestone 2		

6.4 End-to-End Traced Demonstration Walkthrough (Deep Trace of FR-010)

Section 7: Initial Project Risk Register & Uncertainty Management (Output 7)

7.1 Risk Assessment Methodology & Quantitative Exposure Formula (RE = P x I)

7.2 Live Project Risk Register Table (RSK-001 through RSK-010)

Risk ID	Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
RSK-001	Scope Creep	Uncontrolled Change	High	Could create delays and quality degradation	High Priority	Controlled change management	Change register with controlled research and engineering decisions attached	Stakeholder	
RSK-002	Schedule Shortening	Stakeholder Changes required release	low	Could impact quality and feature implementation	Medium Priority	Establish strict deadline that will not be changed	Adjust functional requirements to fit new schedule	The team	

Section 8: Forward Engineering Considerations Register

8.1 The Strategic Engineering Horizon: Thinking Ahead vs. Premature Coding

In strict compliance with Milestone 1 Brief 5, Milestone 1 is an engineering foundation and requirements baseline. It is not an architecture, technology-selection, or implementation milestone. To preserve downstream architectural options and prevent premature design lock-in, the following deliverables are explicitly excluded from Milestone 1:

- Final technology-stack selection (deliberately deferred via ADR-003).
- Final software architecture and component design.
- Detailed database schema and SQL persistence implementation.
- Detailed UI implementation and frontend coding.

- REST API implementation and routing code.
- Design pattern implementation.
- CI pipeline and automated test suite execution.
- Extensive application coding and production deployment.

8.2 Register of 6 Strategic Project-Specific Lifecycle Concerns

8.2.1 FEC-001: Security Architecture & RBAC Perimeter

- **Milestone 1 Context & Rationale:** CivicConnect processes community service requests containing sensitive Personally Identifiable Information (PII)—citizen names, residential addresses, contact numbers, and descriptions of security vulnerabilities. Under South African statutory law (Protection of Personal Information Act, Act 4 of 2013), processing PII mandates strict purpose specification, access restriction, and data integrity safeguards.
- **Downstream Activities Influenced:** Directly dictates Milestone 2 domain entity boundaries, API route authorization middleware, JWT token claim structures, and database row-level security policies.
- **Current Information Gap (M1):** Exact cross-departmental ticket escalation rules (e.g., whether facility supervisors can view municipal police tickets).
- **Risk of Postponing Consideration:** Treating security as an implementation add-on will result in developers constructing monolithic database queries that expose full citizen contact details to third-party maintenance contractors, violating POPIA and requiring a complete database and API rewrite in Milestone 3.
- **Justified Next Action:** Formalize conceptual RBAC roles (Requester, Technician, Supervisor, Admin) in FR-006 and NFR-004. Architect authorization middleware and token validation filters in Milestone 2.

8.2.2 FEC-002: Automated Testability Architecture & Mocking Abstractions

- **Milestone 1 Context & Rationale:** Testability cannot be retrofitted onto an existing codebase. As established by Martin Fowler (2018) and ISO/IEC 25010:2023, automated verification requires modular component architectures featuring loose coupling, dependency inversion, and mockable external interfaces.
- **Downstream Activities Influenced:** Influences Milestone 2 software component design (enforcing Repository and Dependency Injection patterns) and dictates test framework selection (e.g., Jest/Vitest or xUnit) for Milestone 3 automated CI pipelines.
- **Current Information Gap (M1):** The specific assertion libraries and mock database providers compatible with the yet-to-be-selected technology stack.
- **Risk of Postponing Consideration:** If requirements lack verifiable acceptance criteria and components are designed with tightly coupled, direct database calls, the team will be

unable to execute automated unit or integration tests in Milestone 3. Automated quality gates will fail, violating SEN381 assessment criteria.

- **Justified Next Action:** Formulate 100% of Milestone 1 acceptance criteria using formal Gherkin syntax (Given-When-Then). Design decoupled interface abstractions and repository contracts in Milestone 2.

8.2.3 FEC-003: Relational Data Persistence & Audit Table Schema Evolution

- **Milestone 1 Context & Rationale:** A municipal service platform requires absolute accountability. Overwriting database records in place destroys historical evidence and prevents management auditing. The database architecture must capture immutable event logs for every ticket transition (who changed status, when, and why).
- **Downstream Activities Influenced:** Dictates Milestone 2 relational schema normalization, foreign key cascade rules, database migration tooling, and the physical separation of mutable operational tables (ServiceRequests) from append-only audit ledgers (RequestAuditLog).
- **Current Information Gap (M1):** High-volume attachment storage strategy (whether file blobs reside in database columns or external S3-compatible cloud object storage).
- **Risk of Postponing Consideration:** Mutable state schemas implemented hastily in Milestone 3 will overwrite status change timestamps and technician IDs, making it impossible to calculate SLA turnaround metrics (FR-013) and corrupting database history during schema migrations.
- **Justified Next Action:** Baseline the 6-state ticket lifecycle state machine in FR-010 and mandate audit logging in NFR-006. Design normalized schemas and audit tables in Milestone 2.

8.2.4 FEC-004: Environment Parity (Dev, Test, Staging, Prod) & Docker Mirroring

- **Milestone 1 Context & Rationale:** Development, automated CI verification, cloud staging, and live production environments routinely suffer from configuration drift, differences in database versions, and conflicting environment variables. Under 12-Factor App principles (Wiggins, 2017), applications must achieve strict environment parity.
- **Downstream Activities Influenced:** Influences Milestone 2 local development scaffolding (authoring docker-compose.yml declarations), container base image selection, and Milestone 3 CI runner environment configurations.
- **Current Information Gap (M1):** Exact container memory limits and ephemeral port allocation behaviors of target free-tier cloud hosting providers.
- **Risk of Postponing Consideration:** The "works on my machine" failure syndrome. Code executing successfully on a developer's local laptop crashes during the final Milestone 4 live demonstration due to missing cloud environment variables, unseeded databases, or operating system incompatibilities.

- **Justified Next Action:** Establish the 4-tier deployment concept in Section 9. Author reproducible Docker Compose specifications matching staging environments during Milestone 2.

8.2.5 FEC-005: Observability, Structured JSON Logging & Diagnostics

- **Milestone 1 Context & Rationale:** In distributed web applications, runtime errors, unhandled database timeouts, and failed background workers cannot be diagnosed through interactive debuggers. Systems must produce structured, machine-readable telemetry that signals operational health and failure causes without exposing sensitive citizen data.
- **Downstream Activities Influenced:** Influences Milestone 2 global error-handling middleware architecture, application logging formatting (e.g., Pino, Winston, or Serilog), and /health endpoint specifications.
- **Current Information Gap (M1):** The specific cloud log aggregation sink and free-tier retention window available for the deployed staging environment.
- **Risk of Postponing Consideration:** Production tickets fail silently. In Milestone 4, web server processes crash under evaluation queries, returning opaque 500 Internal Server Error responses with zero diagnostic breadcrumbs, destroying reliability scores.
- **Justified Next Action:** Mandate structured error handling in NFR-009. Specify health probe contracts (/health) and JSON logging schemas in Section 9.

8.2.6 FEC-006: Operational Cost & Free-Tier Quota Sustainability

- **Milestone 1 Context & Rationale:** Educational and municipal software projects are strictly bound by resource limits. The project constraint mandates a \$0.00/month hosting expenditure, requiring the platform to remain 100% operational within free-tier compute, database connection, and storage quotas.
- **Downstream Activities Influenced:** Constrains Milestone 2 technology stack selection, database hosting models (e.g., avoiding database services that pause compute after inactivity or charge for egress bandwidth), and asset caching strategies.
- **Current Information Gap (M1):** The 2026 pricing tiers, cold-start latencies, and active connection limits for candidate cloud providers (e.g., Render, [Fly.io](#), Neon, Supabase).
- **Risk of Postponing Consideration:** Architectural designs that rely on high memory footprints (e.g., multi-gigabyte Java Virtual Machines or memory-intensive background queues) exceed free-tier 512MB RAM caps, triggering unexpected account suspensions and fatal deployment failures mid-assessment.
- **Justified Next Action:** Document free-tier hosting limits as a binding constraint in NFR-010. Conduct empirical quota and cold-start evaluations during Milestone 2 stack selection.

8.3 Justified Decision Deferment Defence

None of the six strategic concerns analyzed above are prematurely implemented in Milestone 1. Establishing requirements and architectural boundaries that accommodate these concerns preserves all viable design options. Prematurely selecting libraries or coding endpoints today would represent uninformed speculation. In accordance with professional engineering standards, full commitment is deferred to Milestone 2, where candidate solutions will be evaluated against baselined quality attributes using empirical decision matrices.

Section 9: Engineering Decision Log & Architecture Decision Records (ADRs)

9.1 Master Engineering Decision Table (Context, Trade-offs & Consequences)

Decision Identifier	Title & Core Subject	System Context & Binding Constraints	Evaluated Alternatives	Final Selected Decision	Engineering Rationale & Justification	Trade-offs & Accepted Downstream Risks	Milestone Status
ADR-001	Scope Baseline & Formal Change Control Governance	Academic schedule is fixed (4 milestones); team size is fixed (3 engineers). Uncontrolled feature requests cause schedule slippage and testing cuts.	1. Informal backlog. 2. Rigid zero-change rule. 3. Baselined scope with Appendix E Change Impact Analysis.	Adopt Option 3: Baselined Scope with Formal Appendix E Change Governance.	Protects delivery velocity and preserves high testing quality while providing a controlled mechanism for justified change.	Minor documentation overhead before adopting changes; prevents ad-hoc developer experiments.	APPROVED (Milestone 1)
ADR-002	GitHub Governance, Protected Main & Two-Reviewer Approval Rule	Need to ensure collective code ownership, eliminate unverified commits, and enforce Master Project Brief 9.	1. Direct commits to <code>main</code> . 2. Single-reviewer PR approval. 3. Mandatory two-reviewer approval on protected <code>main</code> .	Adopt Option 3: Protected main branch requiring TWO independent peer reviews.	Complies with institutional standards; ensures 100% of the team inspects every change; eliminates single points of failure.	Increases PR merge turnaround time; requires strict team review SLAs (24h).	APPROVED (Milestone 1)
ADR-003	Deliberate Deferment of Technology Stack Selection to Milestone 2	Committing to frameworks in M1 before analyzing NFRs violates M1 Brief 5 and constitutes premature design lock-in.	1. Lock into Node.js / React in M1. 2. Lock into ASP.NET Core / C# in M1. 3. Formally defer stack selection to M2.	Adopt Option 3: Deliberately defer final stack selection to Milestone 2.	Protects engineering objectivity; ensures stack fit against baselined quality attributes (latency, memory, free-tier hosting).	Delays repository code boilerplate setup to the start of Milestone 2.	APPROVED (Milestone 1)

9.2 Formal Milestone 1 ADRs:

9.2.1 ADR-001: *Scope Baseline and Strict Change Control Governance*

- **Status:** APPROVED
- **Date:** 2026-09-03
- **Decision Owners:** Full Project Team (Chris Fourie, Pandora Greyling, Lisa Verson)
- **Context & Problem Statement:** CivicConnect is engineered by three students under fixed academic deadlines across four milestones. Spontaneous feature additions or informal user feedback introduce scope creep, compressing time available for automated testing and POPIA compliance checks. A controlled governance mechanism is required to freeze committed baseline scope and regulate change.
- **Evaluated Alternatives:**
 - *Alternative 1: Informal Backlog.* Features added dynamically as ideas emerge. (Rejected: High risk of missed deadlines and scope bloat).
 - *Alternative 2: Rigid Zero-Change Policy.* Strict refusal of any modifications. (Rejected: Destroys agility and rejects legitimate stakeholder corrections).
 - *Alternative 3: Controlled Scope Baseline with Appendix E Impact Analysis.* Formally freezing scope; modifications require assessing impact on requirements, architecture, schedule, cost, quality, and risk before authorization. (Selected).
- **Decision Outcome:** Adopted Alternative 3. The 14 Functional Requirements (FR-001 to FR-014) are baselined. Any change must execute a formal Master Project Brief Appendix E Change Impact Analysis.
- **Trade-offs & Downstream Consequences:** Guaranteed schedule predictability and RTM traceability, accepted minor documentation overhead for feature updates. In Milestone 3, this provides the exact audit trail needed for the lecturer/client change scenario.

9.2.2 ADR-002: *GitHub Governance, Protected Main, and Two-Reviewer PR Policy*

- **Status:** APPROVED
- **Date:** 2026-09-03
- **Decision Owner:** Student 3 (Chris Fourie — Systems Architect & Governance Lead)
- **Context & Problem Statement:** Direct commits to main, unreviewed code, and superficial "rubber-stamping" of pull requests create merge conflicts, broken builds, and knowledge silos. Per Master Project Brief 9, GitHub is an engineering control environment requiring configuration management and collective ownership.
- **Evaluated Alternatives:**
 - *Alternative 1: Direct Commits to Main.* Unrestricted pushing. (Rejected: Fatal regression risk and zero audit trail).

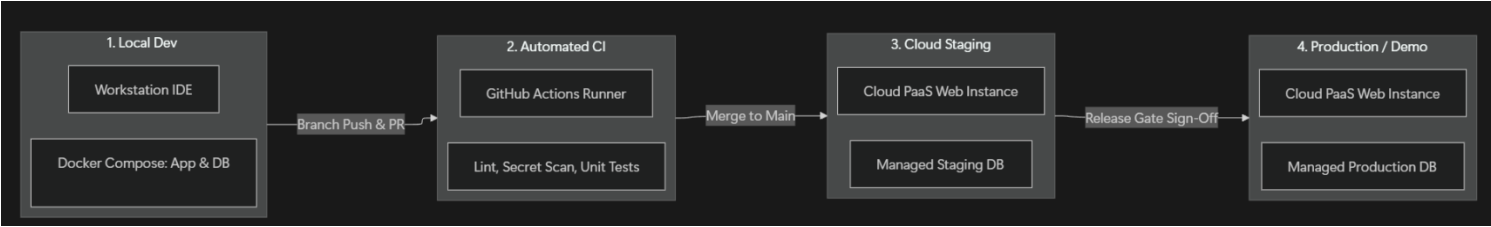
- *Alternative 2: Single-Reviewer PR Approval.* Pull requests require one peer sign-off. (Rejected: Leaves 33% of the team unaware of code changes).
- *Alternative 3: Protected Main with Mandatory Two-Reviewer Approval.* Direct commits blocked; all changes enter via PR requiring approvals from BOTH other team members. (Selected).
- **Decision Outcome:** Adopted Alternative 3. Main branch protection rules enforced in GitHub settings. Every change requires an issue-linked branch (feat/FR-xxx, docs/xxx) and approvals from two independent team members. Self-approvals are blocked.
- **Trade-offs & Downstream Consequences:** Enforces Linus's Law ("Given enough eyeballs, all bugs are shallow") and ensures all 3 students can defend any artifact during oral defence. Accepted trade-off of increased PR turnaround latency, mitigated by a 24-hour review SLA.

9.2.3 ADR-003: *Justified Deferment of Technology Stack Selection to Milestone 2*

- **Status:** APPROVED
- **Date:** 2026-09-03
- **Decision Owner:** Full Team (Authored by Chris Fourie — Systems Lead)
- **Context & Problem Statement:** Committing to web frameworks, programming languages, or database engines in Milestone 1 is an engineering anti-pattern driven by personal developer familiarity rather than objective requirements. It violates Milestone 1 Brief 5 and constitutes premature design lock-in.
- **Evaluated Alternatives:**
 - *Alternative 1: Lock into Node.js / Express / React / PostgreSQL in M1.* (Rejected: Premature lock-in without evaluating memory and cold-start latency).
 - *Alternative 2: Lock into [ASP.NET](#) Core Web API / C# / SQL Server in M1.* (Rejected: Premature lock-in without evaluating free-tier Linux hosting limits).
 - *Alternative 3: Deliberately Defer Technology Stack Selection to Milestone 2.* (Selected).
- **Decision Outcome:** Adopted Alternative 3. Milestone 1 establishes problem analysis and baselined quality attributes (NFR-001 to NFR-010). Milestone 2 will evaluate candidate stacks against these baselines using an objective weighted decision matrix and technical proof-of-concept spikes.
- **Trade-offs & Downstream Consequences:** Protects architectural objectivity and prevents costly framework migrations during Milestone 3. Accepted trade-off of delaying project boilerplate setup to the start of Milestone 2.

Section 10: Deployment, Environments & Operational Readiness Concept

10.1 The Four-Tier Environment Separation Model



Environment Tier	Target Runtime Infrastructure	Primary Engineering Purpose	Configuration & Isolation Controls	Data Persistence Strategy
Tier 1: Local Development	Local developer workstation running Docker Desktop.	Rapid inner-loop development, interactive debugging, and local unit test execution.	Environment variables loaded via. <code>env.local</code> ; hot-reloading enabled; verbose debugging logs active.	Ephemeral local containerized database pre-seeded with synthetic development fixtures.
Tier 2: Automated CI	Headless, ephemeral Ubuntu virtual runners on GitHub Actions.	Clean-room verification of every PR: automated linting, secret scanning, and automated unit/integration tests.	Strict sandbox; zero persistent state; environment variables injected dynamically from GitHub Actions Secrets.	In-memory or ephemeral container database created and destroyed per test run.
Tier 3: Cloud Staging	Cloud PaaS (e.g., Render, Fly.io) with managed database (Neon / Supabase).	Production-like integration verification, secrets validation, performance benchmarking, and smoke testing.	Production-mirror runtime; HTTPS enforced; secure cloud environment variables; automated deploy on <code>main</code> merge.	Dedicated staging database containing sanitized synthetic data mirroring production schema.
Tier 4: Production / Demo	Hardened cloud PaaS web service and production-tier database instance.	Stable, locked demonstration platform for stakeholder evaluation and	Optimized production build; debugging endpoints disabled; rate limiting enforced; strict	Production database with persistent, append-only audit tables and automated daily backups.

		oral engineering defence.	POPIA access logging.	
--	--	------------------------------	--------------------------	--

10.2 Environment Parity & Local Containerized Replication

- **Strict Separation of Code and Config (12-Factor App):** In strict adherence to The Twelve-Factor App methodology (Wiggins, 2017), all configuration parameters that vary across deployment environments (database connection strings, server port bindings, session encryption secrets, and JWT signing keys) are injected exclusively via environment variables.
- **Zero Secrets in Git Commits:**
 - Plaintext credentials, passwords, API tokens, and private encryption keys are prohibited from entering version control.
 - The repository root provides a documented .env.example file defining required configuration keys paired with non-sensitive documentation defaults.
 - Local .env and .env.local files are explicitly ignored via .gitignore.
 - Automated pre-commit hooks and GitHub Actions workflows incorporate automated secret-detection scanners (e.g., Gitleaks) to detect and block accidental credential commits.

10.3 Deployment vs Release Distinction & Rollback Strategy

In strict accordance with Master Project Brief 17, deployment and release are treated as distinct, decoupled engineering lifecycle activities:

- **Deployment:** Represents the automated continuous integration delivery of compiled software binaries and configuration assets to cloud staging environments via GitHub Actions. Deployment validates technical compilation, dependency resolution, and integration compatibility without exposing functionality to community users.

- **Release:** Represents the formal administrative authorization and feature activation for end-users, governed by milestone sign-off gates and business verification.

Rollback & Disaster Recovery Concept:

To eliminate operational downtime and prevent data corruption during failed deployments:

- Cloud staging and production environments utilize immutable container image tags, enabling instantaneous one-click rollback to the preceding stable commit SHA via cloud PaaS deployment hooks.

- Database schema changes are strictly governed by paired Up/Down migration scripts. Before applying any schema modification in staging or production, a verified rollback script is executed against an ephemeral database replica, ensuring zero persistent data loss in the event of an emergency reversion.

10.4 Operational Health Probes & Structured JSON Telemetry

To fulfil **Master Project Brief** regarding system observability and operational readiness:

- **Liveness & Readiness Health Endpoint:** The application will expose a lightweight HTTP GET endpoint at /health. This endpoint executes a fast database ping to verify connectivity and runtime process health, returning HTTP 200 OK without placing locks on operational tables:

```
{
  "status": "healthy",
  "timestamp": "2026-09-03T10:15:30Z",
  "version": "1.0.0",
  "uptime_seconds": 84210,
  "dependencies": {
    "database": {
      "status": "connected",
      "latency_ms": 14
    }
  }
}
```

If the database connection fails, the endpoint immediately returns HTTP 503 Service Unavailable, enabling cloud PaaS orchestrators to trigger automated restarts or reroute traffic.

- **Structured JSON Error Logging:** All runtime exceptions and state-machine transitions are emitted to stdout as single-line, structured JSON objects:

```
{  
  "timestamp": "2026-09-03T10:16:05Z",  
  "level": "ERROR",  
  "correlation_id": "c8f1e4b2-8491-4d7a",  
  "component": "StateTransitionEngine",  
  "event": "INVALID_STATE_TRANSITION",  
  "request_id": "REQ-2026-0104",  
  "from_state": "SUBMITTED",  
  "attempted_to_state": "RESOLVED",  
  "actor_id": "USR-8821",  
  "message": "Direct transition from SUBMITTED to RESOLVED rejected."  
}
```

To guarantee compliance with South African POPIA regulations, all log serialization formatters apply automated sanitization filters that redact passwords, citizen telephone numbers, national identity numbers, and personal physical addresses before output

Section 11: Milestone 1 Baseline Sign-Off Gate & Governance Audit

11.1 Master Project Brief Appendix D Verification Table

Evaluation Attribute	Verified Engineering Finding	Baseline Gate Status
Project Identifier	CivicConnect: Community Service Request Management Platform	VERIFIED
Baseline Type	Milestone 1 — Engineering Foundation & Requirements Baseline	VERIFIED
Document Baseline Version	v1.0 (Controlled Product State)	VERIFIED
Sign-Off Date	2026-09-09	VERIFIED
Scope Reviewed?	YES — All 14 committed Functional Requirements (FR-001 to FR-014) verified. Explicit exclusions (Commercial Billing, Real-	PASSED

	Time Vehicle GPS, Multi-Tenant SaaS) defended.	
Requirements & Traceability Checked?	YES — Requirements Traceability Matrix (RTM v1.0) established with stable IDs, stakeholder sources, and verifiable Gherkin acceptance criteria.	PASSED
Risk Review Completed?	YES — 10 project risks evaluated with root causes, probability, impact, exposure, mitigations, and contingencies. #1 Critical Risk (RSK-001: Scope Creep) defended.	PASSED
Repository & Governance Controls Checked?	YES — GitHub repository established with protected main branch, gitignore, PR template, and mandatory two-reviewer approval rule fully operational.	PASSED
FORMAL GATE OUTCOME	ACCEPTED	ACCEPTED

11.2 Gate Review Summary & Readiness Declaration

Our CivicConnect engineering team has fulfilled all required Milestone 1 deliverables. The team has demonstrated clear engineering intent, disciplined scope boundaries, rigorous repository governance, and responsible AI accountability. The baseline PED v1.0 provides an auditable, controlled engineering foundation to proceed directly to Milestone 2 (Architecture, Design & Engineering Decisions).

11.3 Formal Sign-Off Record & Milestone Gate Outcome (**ACCEPTED**)

- **Milestone Gate Decision: ACCEPTED**
- **Sign-Off Authority:** Full Engineering Team
 - Lead Requirements Analyst: *Pandora Greyling* (Signed: 2026-09-09)
 - Quality & Risk Manager: *Lisa Verson* (Signed: 2026-09-09)
 - Systems Architect & Governance Lead: *Chris Fourie* (Signed: 2026-09-09)

Section 12: Academic Reference List

12.1 International Software Engineering Standards (IEEE, ISO/IEC)

- **IEEE Computer Society.** (2014). *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide V3.0)*. Piscataway, NJ: IEEE Computer Society Press.
- **ISO/IEC.** (2023). *ISO/IEC 25010:2023: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. Geneva: International Organization for Standardization.
- **W3C.** (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C Recommendation. Available at: <https://www.w3.org/TR/WCAG21/> (Accessed: 1 September 2026).

12.2 Architectural & Methodological Literature (Bass et al., Boehm, Fowler)

- **Bass, L., Clements, P. and Kazman, R.** (2021). *Software Architecture in Practice*. 4th edn. Boston, MA: Addison-Wesley Professional.
- **Boehm, B.W.** (1981). *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice-Hall.
- **Boehm, B. and Basili, V.R.** (2001). 'Software Defect Reduction Top 10 List', *Computer*, 34(1), pp. 135–137. doi: 10.1109/2.962984.
- **Fowler, M.** (2018). *Refactoring: Improving the Design of Existing Code*. 2nd edn. Boston, MA: Addison-Wesley.
- **Humble, J. and Farley, D.** (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ: Addison-Wesley.
- **Kleppmann, M.** (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O'Reilly Media.
- **Nygaard, M.T.** (2018). *Release It!: Design and Deploy Production-Ready Software*. 2nd edn. Raleigh, NC: Pragmatic Bookshelf.
- **Wiggins, A.** (2017). *The Twelve-Factor App*. Available at: <https://12factor.net/> (Accessed: 1 September 2026).
- **Zdun, U., Capilla, R., Tran, H. and Zimmermann, O.** (2013). 'Sustainable Architectural Design Decisions', *IEEE Software*, 30(6), pp. 46–53. doi: 10.1109/MS.2013.127.
- **Zimmermann, O., Wegmann, L., Koziol, H. and Goldschmidt, T.** (2015). 'Architectural Decision Guidance Across Projects', *IEEE Software*, 32(1), pp. 14–19. doi: 10.1109/MS.2015.19.

12.3 Statutory Legislation & Security Frameworks (POPIA, NIST, OWASP)

- **NIST (National Institute of Standards and Technology).** (2022). *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities*. NIST Special Publication 800-218. Gaithersburg, MD: U.S. Department of Commerce. doi: 10.6028/NIST.SP.800-218.
- **OWASP Foundation.** (2021). *OWASP Top 10: 2021 — The Ten Most Critical Web Application Security Risks*. Available at: <https://owasp.org/Top10/> (Accessed: 1 September 2026).

- **Republic of South Africa.** (2013). *Protection of Personal Information Act (Act No. 4 of 2013)*. Government Gazette, 581(37067). Cape Town: Government Printer.

Section 13: Appendices

Appendix A: Master Project Brief Appendix D Baseline Sign-Off Table

Appendix B: Requirements Traceability Matrix (RTM v1.0)

Appendix C: Project Risk Register (RSK-001 to RSK-010)

Appendix D: Team Working Agreement & Collaboration SLA

This Team Working Agreement establishes the professional norms, communication cadences, pull request turnaround service level agreements (SLAs), and dispute escalation protocols for CivicConnect Group E. All three registered team members are bound by this agreement in accordance with Master Project Brief.

Pull Request Review & Turnaround SLAs

- Minimum Two Approvals: Every pull request entering protected development or main branches requires two independent approvals from team members other than the author.
- Review Turnaround SLA: Reviewers must inspect and provide meaningful comments or approvals on open PRs within 24 hours of submission.
- Rubber-Stamping Ban: Reviews consisting solely of empty approvals are strictly invalid. Reviewers must verify:
 1. Alignment with baselined requirements (FR-xxx / NFR-xxx).
 2. Absence of plaintext credentials or private keys.
 3. Clean separation of concerns with zero unnecessary complexity.
 4. Updating of traceability links and documentation records.
- Rework Protocol: If changes are requested, the PR author must push updates or schedule a clarification call within 24 hours.

Dispute Resolution & Individual Accountability Escalation

- Technical Disagreements: In the event of competing technical preferences, decisions will be evaluated against baselined constraints (schedule, cost, testability, POPIA security) using a formal Weighted Decision Matrix. If a consensus cannot be reached within 48 hours, the designated role owner holds the deciding vote, documented in an ADR.

- Contribution & Accountability Non-Performance: If a team member fails to attend scheduled syncs or deliver assigned milestone commitments for greater than 48 hours without approved academic exemption, the incident is formally logged in the milestone retrospective and escalated to the module lecturer per SEN381 institutional guidelines.

Appendix E: Responsible AI Usage Register v1.0 & Case Critique

Date	Student	AI Tool & Model	Engineering Task	Material AI Contribution	Human Verification Applied	Final Decision	Identified Issues, Hallucinations & Rejections
2026-09-01	Chris Fourie	Claude 3.5 Sonnet	Document Scaffolding	Drafted initial text for project baseline structure.	Verified against Master Project Brief and institutional standards.	MODIFIED	AI produced generic textbook definitions of software engineering; stripped out generic prose and focused purely on CivicConnect system controls.
2026-09-02	Chris Fourie	Gemini 3.1 Pro	Brainstorming Forward Engineering	Proposed candidate concerns for Forward Engineering.	Audited proposed concerns against Master Brief 4 and Boehm's Cost of Change curve.	REJECTED & REPLACED	AI suggested "Kubernetes Cluster Ingress" and "Multi-Region Distributed Caching" for M1. Completely rejected as unjustified over-engineering; replaced with Docker Compose parity and free-tier cloud quotas.
2026-09-02	Chris Fourie	ChatGPT (GPT-4o)	Drafting ADR-002	Drafted Pull Request governance policy.	Audited against Master Project Brief	MODIFIED	AI proposed a single-reviewer PR policy. Rejected and corrected to mandate TWO independent peer approvals from non-author team members (100% peer review).
2026-09-03	Chris Fourie	Google Antigravity	Reference List Verification	Formatted Harvard reference entries.	Manually cross-referenced author names, publication years, DOIs, and editions on Google Scholar.	VERIFIED & ACCEPTED	Verified all 16 sources on Google Scholar; corrected ISO/IEC 25010 to cite the current 2023 edition.

In accordance with Master Project Brief, Artificial Intelligence is integrated throughout the SEN381 project as an engineering assistant. AI output is never authoritative evidence and does not transfer professional or legal accountability away from the engineering team.

During Milestone 1, the engineering team applied a strict 5-stage human-in-the-loop verification pipeline: Prompt Specification -> Output Generation -> Primary Domain Audit -> Correction/Rejection -> Baseline Commitment. Two critical cases highlight the necessity of this human oversight:

Case 1: Rejection of Over-Engineered Cloud Ingress (2026-09-02)

When prompted to explore candidate forward engineering concerns for CivicConnect, Gemini 3.1 Pro proposed implementing a distributed Kubernetes cluster with multi-region ingress controllers and Redis cache replication. During the human audit, this recommendation was completely rejected. While technically sophisticated, introducing a multi-node Kubernetes architecture directly violates our binding R0.00/month free-tier budget constraint (NFR-010) and introduces severe operational complexity that threatens our 3-person team capacity. It was replaced with a lightweight Docker Compose local parity model.

Case 2: Rectification of Single-Reviewer Governance Vulnerability (2026-09-02)

When generating initial branch protection policies for ADR-002, ChatGPT (GPT-4o) generated a standard corporate configuration requiring a single peer approval. In a 3-person team, a single-reviewer model leaves 33% of the team completely blind to merged changes, creating technical knowledge silos that fatalize individual engineering defence examinations. The team modified the policy to mandate TWO independent peer approvals, enforcing 100% collective codebase ownership across all three students.

Student Attestation:

The team confirms that all AI contributions were independently verified against primary software engineering standards (IEEE SWEBOK, ISO/IEC 25010, POPIA Act 4 of 2013). We assume full personal, academic, and professional accountability for all submitted artefacts.